

Benemérita Universidad Autónoma de Puebla

Facultad de Ciencias de la Computación

Asignatura: Minería de Datos

Profesora: Bernábe Loranca María Beatriz

NRC:55288

Manual Técnico

Alumnos:

Bernardo Román Hernández 202157144

Brandon Rodrigo Méndez Méndez 202130905

Valeria Trevera Arcos 202163449

Fecha: 17 de Mayo de 2024

Periodo: Primavera 2024

Se deben importar las siguientes librerías:

pandas: Esta es una biblioteca esencial en Python para la manipulación y análisis de datos que nos permite trabajar con datos tabulares similares a las hojas de cálculo de manera eficiente.

re: Esta es una biblioteca estándar de Python para trabajar con expresiones regulares. Las expresiones regulares son útiles para realizar búsquedas y manipulaciones avanzadas de texto. se utiliza para limpiar texto eliminando caracteres no deseados.

```
import pandas as pd
import re
```

NumPy es una biblioteca de Python utilizada para realizar cálculos numéricos. En este caso, se utiliza para trabajar con matrices y arrays multidimensionales.

itertools es un módulo de Python que proporciona funciones para trabajar con iteradores. En este caso, se utiliza para generar todas las combinaciones posibles de pares de etiquetas en la matriz de confusión.

Estas bibliotecas son esenciales para la correcta visualización de la matriz de confusión y la impresión de sus valores en la representación gráfica.

```
import numpy as np
import itertools
```

Matplotlib es una biblioteca de trazado en 2D de Python que produce figuras de alta calidad en una variedad de formatos y entornos. Proporciona una forma sencilla de crear visualizaciones como gráficos de líneas, gráficos de dispersión, histogramas, etc.

matplotlib.pyplot es un submódulo de Matplotlib que proporciona una interfaz similar a la de MATLAB para crear gráficos. Con este submódulo, se pueden crear gráficos y visualizaciones de datos de manera rápida y sencilla.

```
import matplotlib.pyplot as plt
```

Se importa la función TSNE del módulo manifold de la biblioteca scikit-learn.

sklearn.manifold

Este módulo de scikit-learn proporciona herramientas para la reducción de dimensionalidad y la visualización de datos de alta dimensionalidad en un espacio de menor dimensión.

TSNE

T-distributed Stochastic Neighbor Embedding (t-SNE) es un algoritmo de reducción de dimensionalidad no lineal ampliamente utilizado para la visualización de datos de alta dimensión en un espacio de dos o tres dimensiones.

```
from sklearn.manifold import TSNE
```

Se importa varias clases y funciones de scikit-learn, una biblioteca popular para aprendizaje automático en Python.

sklearn.feature_extraction.text.CountVectorizer

CountVectorizer es una clase de scikit-learn que convierte una colección de documentos de texto en una matriz de recuentos de términos. Cada fila de la matriz representa un documento y cada columna representa una palabra del vocabulario. Los valores en la matriz son los recuentos de ocurrencias de cada palabra en cada documento.

sklearn.model_selection.train_test_split

train_test_split es una función de scikit-learn que divide un conjunto de datos en subconjuntos de entrenamiento y prueba. Esto es útil para evaluar el rendimiento de un modelo de aprendizaje automático en datos no vistos.

sklearn.naive_bayes.MultinomialNB

MultinomialNB es una clase de scikit-learn que implementa el clasificador de Naive Bayes multinomial. Es adecuado para clasificar datos que se pueden representar como recuentos

```
from sklearn.feature_extraction.text import CountVectorizer
from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import MultinomialNB
```

de palabras o frecuencias de palabras, como en el caso de la clasificación de texto.

importa varias funciones de scikit-learn que son útiles para evaluar el rendimiento de modelos de clasificación:

`sklearn.metrics.accuracy_score`

`accuracy_score` es una función que calcula la precisión del modelo, que es la proporción de muestras clasificadas correctamente sobre el total de muestras. Se utiliza para evaluar la exactitud del modelo en la clasificación.

`sklearn.metrics.confusion_matrix`

`confusion_matrix` es una función que calcula una matriz de confusión, que es una tabla que describe el rendimiento del modelo de clasificación. Muestra el número de verdaderos positivos, verdaderos negativos, falsos positivos y falsos negativos.

`sklearn.metrics.classification_report`

`classification_report` es una función que genera un informe de clasificación, que incluye métricas como precisión, recall, puntuación F1 y soporte para cada clase en el conjunto de datos. Proporciona una visión general del rendimiento del modelo para cada clase de manera detallada.

```
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report
```

Se importa la clase `GridSearchCV` del módulo `model_selection` de scikit-learn.

`sklearn.model_selection.GridSearchCV`

`GridSearchCV` es una clase que realiza una búsqueda exhaustiva sobre una cuadrícula de parámetros especificada para un estimador. Es útil para encontrar la combinación óptima de parámetros para un modelo mediante validación cruzada. La búsqueda se realiza evaluando el rendimiento del modelo para cada combinación de parámetros y seleccionando aquellos que maximizan (o minimizan) una métrica de rendimiento especificada.

```
from sklearn.model_selection import GridSearchCV
```

Se debe verificar la correcta instalación de las librerías para un correcto funcionamiento del sistema.

Cargar Archivos Excel

`pd.read_excel("archivo.xlsx")`: Esta función de pandas carga datos desde un archivo Excel y los almacena en un DataFrame.

`df_negativa`: DataFrame que contiene palabras negativas, cargado desde "Palabras_Negativas.xlsx".

`df_positiva`: DataFrame que contiene palabras positivas, cargado desde "Palabras_Positivas.xlsx".

`df_tweets`: DataFrame que contiene tweets clasificados, cargado desde "XochitlGalvezClasificados.xlsx".

```
# Cargar archivos Excel de Xochitl
try:
    df_negativa = pd.read_excel("Palabras_Negativas.xlsx")
    df_positiva = pd.read_excel("Palabras_Positivas.xlsx")
    df_tweets = pd.read_excel("XochitlGalvezClasificados.xlsx")
```

Limpieza de Textos en Tweets

`df_tweets['tweets limpios']` se encarga de seleccionar la columna tweets limpios del DataFrame `df_tweets`.

`.apply(lambda x: ...)` aplica una función lambda a cada elemento de la columna.

`lambda x: ...` define una función anónima (lambda) que toma un argumento `x`.

`x.lower()` convierte el texto de `x` a minúsculas.

`re.sub(r'^a-zA-Z\s', '', x)` usa una expresión regular para reemplazar cualquier carácter que no sea una letra o un espacio con una cadena vacía, eliminando así caracteres no alfabéticos.

El resultado de esto es una nueva columna `tweets_limpios` con el texto limpiado.

```
# Convertir el texto a minúsculas y eliminar caracteres no alfabéticos mediante expresiones regulares
df_tweets['tweets_limpios'] = df_tweets['tweets limpios'].apply(lambda x: re.sub(r'^a-zA-Z\s', '', x.lower()))
```

Imprimir las Primeras Filas de Cada DataFrame

`print("Texto")` imprime el texto especificado en la consola.

`df_negativa.head()`, `df_positiva.head()`, `df_tweets.head()` muestran las primeras cinco filas de cada DataFrame.

Esto es útil para inspeccionar rápidamente los datos y verificar que se han cargado correctamente.

```
# Ejemplo de operación: imprimir las primeras filas de cada DataFrame
print("Primeras filas de BibliotecaFinalNegativa.xlsx:")
print(df_negativa.head())

print("\nPrimeras filas de BibliotecaFinalPositiva.xlsx:")
print(df_positiva.head())

print("\nPrimeras filas de Sb_tweets_limpios.xlsx:")
print(df_tweets.head())
```

Manejo de Excepciones

`except FileNotFoundError:` captura el error específico de que uno o más archivos no se encuentran.

Muestra un mensaje de error para que el usuario verifique los nombres de los archivos y su ubicación.

`except Exception as e:` captura cualquier otro tipo de error que pueda ocurrir durante la ejecución del bloque `try`.

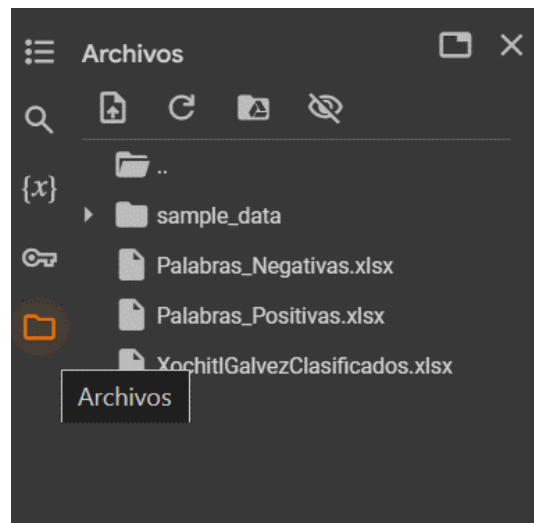
Muestra un mensaje de error con la descripción del mismo (`e`), lo que ayuda a identificar y corregir el problema.

```
except FileNotFoundError:
    print("No se pudo encontrar uno o más archivos. Asegúrate de que los nombres de los archivos sean correctos y que estén en el directorio actual.")
except Exception as e:
    print("Ocurrió un error al cargar los archivos:", e)
```

Cargar Archivos para Análisis

En la barra lateral izquierda de Colab, deberá de hacer clic en el ícono de la carpeta para abrir el panel de archivos. Se hace clic en el ícono de subir archivos (una flecha hacia arriba) o seleccionar "Subir" desde las opciones.

Se selecciona el archivo que deseas subir desde tu computadora. Una vez que se hayas subido el archivo, aparecerá en el sistema de archivos de Colab, y se podrá acceder a él usando las rutas de archivo.



Se debe verificar una vez cargados los archivos que los nombre de estos coincidan en la parte de Cargar los archivos, si no se deberá cambiar el nombre por el que tenga cada uno de los archivos cargados para un correcto funcionamiento

```
# Cargar archivos Excel de Xochitl
try:
    df_negativa = pd.read_excel("Palabras_Negativas.xlsx")
    df_positiva = pd.read_excel("Palabras_Positivas.xlsx")
    df_tweets = pd.read_excel("XochitlGalvezClasificados.xlsx")
```

Definición de Palabras Positivas

palabras_positivas: Esta lista contiene palabras que generalmente tienen connotaciones positivas y se utilizan para expresar apoyo, confianza, y emociones positivas.

Uso: Estas palabras las usamos para identificar tweets o textos que expresan sentimientos positivos

```
# Definición de palabras positivas y negativas iniciales
palabras_positivas = ['apoyo', 'vamos', 'ganar', 'contigo', 'seguridad', 'bendiciones', 'confianza', 'fuerza', 'arriba',
'ganadora', 'buena', 'mejor', 'chingon', 'arriba', 'vencedora', 'futura presidenta', 'futura presidente',
'preferencia', 'futuro', 'bueno', 'juntos', 'trabajo', 'cambio', 'bienestar', 'ayuda', 'esperanza',
'construyendo', 'transformacion', 'capaz', 'justa', 'diplomática', 'científica', 'ingeniera', 'humilde',
'bravo', 'con todo', '100', 'vencedores', 'vencedora', 'inteligente', 'mejor opcion',
'respetar', 'proceso', 'Líderes', 'apoyar', 'Esperanza', 'gana', 'Apoyo', 'Bien', 'ganarse', 'Transformación',
'Ganas', 'Feliz', 'amor', 'top', 'Valiente', 'lider', 'visionaria', 'visionario', 'comprometido', 'eficaz',
'dedicado', 'valiente', 'justo', 'honesto', 'comunicativo', 'persuasivo', 'carismatico', 'carismática',
'comunicativa', 'honesta', 'responsable', 'respetuoso', 'transparente', 'determinada', 'detererminado',
'inclusivo', 'inclusiva', 'diplomático', 'audaz', 'innovador', 'emprendedor', 'apasionada', 'prudente',
'apasionado', 'decidida', 'decidido', 'tenaz', 'modesto', 'modesta', 'sagaz', 'familiar', 'mayor', 'creado',
'empleo', 'festejar', 'crear', 'bien', 'propósito', 'trabajando', 'desarmar', 'proponen', 'defensores',
'soberanía', 'ALEGRÍA', 'CAMBIO', 'progreso', 'apoyan', 'Protección', 'iniciativa', 'defender', 'servir',
'Poder', 'vida', 'protagonista', 'Respaldo', 'promueven', 'gracias', 'Bienvenido', 'inversión', 'titular',
'apoyando', 'Gracias', 'perfecta', 'ley', 'ideal', 'felices', 'Amigos', 'derecho', 'rescatar', 'protege',
'joya', 'esencial', 'fuerte', 'familia', 'APOYANDO', 'gusta', 'poder', 'Respeto'
]
```

Definición de Palabras Negativas

palabras_negativas: Esta lista contiene palabras que generalmente tienen connotaciones negativas y se utilizan para expresar descontento, insultos, y emociones negativas.

Uso: Estas palabras las usamos para identificar tweets o textos que expresan sentimientos negativos hacia una persona, evento o situación.

```
palabras_negativas = arreglo_palabras = [
'denunciaron', 'robo', 'Insatisfacción',
'vividora', 'tristeza', 'Rechazo', 'estúpida', 'pendeja', 'babosa', 'idiota', 'perro', 'perra', 'ratera',
'lamebotas', 'gata', 'verga', 'argüendera', 'fodonga', 'mamona', 'mitotera', 'pinche', 'chingada', 'amarrete',
'cascado', 'chocha', 'huevos', 'pendejear', 'pendejos', 'perdedora', 'basura', 'asquerosa', 'cobarde', 'ridícula',
'dientona', 'flaca', 'ladilla', 'gusana', 'adesio', 'analfabeta', 'boba', 'zorra', 'creida', 'imbecil', 'chencha',
'subnormal', 'cabrona', 'Narcos', 'burla', 'burra', 'ratero', 'malandro', 'malandra', 'creido', 'chencho', 'pendejo',
'putizima', 'mentiroso', 'mentirosa', 'desgraciado', 'desgraciada', 'racista', 'clasista', 'presumida', 'mamada',
'mamadas', 'fea', 'feo', 'horrible', 'asno', 'rateros', 'idioacia', 'mantecato', 'mantecata', 'mazacote',
'convenciera', 'convenenciero', 'convenecieros', 'traidores', 'traidora', 'traidor', 'payaso', 'payasa',
'conservador', 'infames', 'miserable', 'jodido', 'jodida', 'empobresida', 'empobresido', 'petulante', 'acusada',
'violar', 'encubrir', 'ocultar', 'corrupción', 'menos', 'afectados', 'duda', 'ignorancia', 'condena', 'Traiciona',
'secta', 'AMAFIADOS', 'limitar', 'denuncian', 'aburrida', 'presumido', 'destruyendo', 'cobardía', 'autoritarios', 'mal',
'traicionado', 'desmedida', 'castigo', 'robándole', 'denunciando', 'riesgo', 'enemiga', 'retroceder', 'prepotencia',
'mata', 'golpes', 'pierde', 'COLUDIDOS', 'corruptos', 'malversaron', 'estúpida', 'coludido', 'violador', 'ataque',
'mercenarios', 'oponen', 'amenazan', 'violencia', 'pobre', 'indefenso', 'queja', 'narco', 'peligro', 'desaparecer',
'burro', 'criticar', 'deplorable', 'infamia', 'mierdero', 'mentir', 'violó', 'miedo', 'mataron', 'delincuentes',
'transa', 'baleado', 'INUTIL', 'incapaz', 'desgracia', 'TONTA', 'expulsado', 'Loco', 'amenaza', 'insultos', 'abucheos',
'decline', 'Balacera', 'dudas', 'ratas', 'daño', 'muerte', 'corrupta', 'inseguridad', 'crisis', 'hambre', 'ratota',
'triste', 'provocación', 'perder', 'hartos', 'Culpable', 'crimen'
]
```

Llenado de Valores Faltantes en la Columna "conteo"

Esta parte llena los valores faltantes en la columna "conteo" de los DataFrames df_negativa y df_positiva con ceros. Estos DataFrames los utilizamos para almacenar las palabras negativas y positivas respectivamente, junto con su conteo de ocurrencias en los tweets.

Uso: Garantizar que todas las palabras tengan un conteo inicial de cero para evitar errores al incrementar sus conteos durante el análisis de los tweets.

Método: El método fillna() se utiliza para rellenar los valores faltantes en la columna "conteo" con ceros. El argumento inplace=True actualiza directamente el DataFrame original en lugar de devolver una copia modificada.

```
df_negativa["conteo"].fillna(0, inplace=True)
df_positiva["conteo"].fillna(0, inplace=True)
```

Bucle de Iteración sobre los Tweets

Este bucle itera sobre cada fila del DataFrame df_tweets, que contiene los tweets limpios.

Uso: Nos permite procesar cada tweet individualmente para identificar las palabras positivas y negativas en ellos.

Método: Para cada tweet, se accede al texto limpio del tweet utilizando row["tweets limpios"]. Luego, se divide el texto en palabras individuales utilizando el método split(), lo que crea una lista de palabras.

```
for index, row in df_tweets.iterrows():

    tweet_text = row["tweets limpios"]

    palabras = tweet_text.split()
```

Búsqueda de Palabras en las Listas de Palabras Positivas y Negativas

Este bucle nos ayuda a itera sobre cada fila del DataFrame `df_tweets`, que contiene los tweets limpios.

Uso: Nos permite procesar cada tweet individualmente para identificar las palabras positivas y negativas en ellos.

Método: Para cada tweet, se accede al texto limpio del tweet utilizando `row["tweets limpios"]`. Luego, se divide el texto en palabras individuales utilizando el método `split()`, lo que crea una lista de palabras.

```
for palabra in palabras:
    if palabra in df_negativa["palabra"].values:
        palabras_negativas.append(palabra)
        #df_negativa.loc[df_negativa["palabra"] == palabra, "conteo"] += 1

    if palabra in df_positiva["palabra"].values:
        palabras_positivas.append(palabra)
        #df_positiva.loc[df_positiva["palabra"] == palabra, "conteo"] += 1
```

Impresión de Resultados

Este parte nos ayuda a imprimir las palabras positivas y negativas encontradas en los tweets.

Uso: Nos proporciona una salida legible para mostrar las palabras identificadas en los tweets y su clasificación como positivas o negativas.

Método: Utiliza la función `print()` para imprimir las listas `palabras_negativas` y `palabras_positivas`, que contienen las palabras identificadas en los tweets. Se separan las palabras negativas y positivas para una mejor visualización.

```
print("Primeras filas de BibliotecaFinalNegativa.xlsx:")
print(palabras_negativas)

print("\nPrimeras filas de BibliotecaFinalPositiva.xlsx:")
print(palabras_positivas)
```

Eliminación de Palabras Repetidas y Conteo de Ocurrencias

Esta parte del código elimina las palabras repetidas de las listas de palabras positivas y negativas, y luego cuenta las ocurrencias de cada palabra.

Uso: Nos permite obtener un recuento de cuántas veces aparece cada palabra en los tweets.

Método:

set(): Convierte la lista de palabras en un conjunto para eliminar duplicados.

list(): Convierte el conjunto nuevamente en una lista para mantener el orden original de las palabras.

El uso de un dict comprehension permite crear un diccionario donde las claves son las palabras y los valores son sus ocurrencias en la lista original.

```
# Eliminar palabras repetidas de las listas y contar las ocurrencias
palabras_positivas_sin_repetir = list(set(palabras_positivas))
palabras_negativas_sin_repetir = list(set(palabras_negativas))

conteo_palabras_positivas = {palabra: palabras_positivas.count(palabra) for palabra in palabras_positivas_sin_repetir}
conteo_palabras_negativas = {palabra: palabras_negativas.count(palabra) for palabra in palabras_negativas_sin_repetir}
```

Creación de DataFrames Actualizados

Este bloque de código crea DataFrames actualizados que contienen las palabras positivas y negativas sin repetir junto con su conteo de ocurrencias.

Uso: Proporcionar una estructura de datos más organizada para analizar y visualizar las palabras y sus conteos.

Método: Utiliza el constructor `pd.DataFrame()` de pandas para crear DataFrames a partir de los diccionarios de palabras y sus conteos. Cada DataFrame tiene dos columnas: "palabra" para la palabra en sí y "conteo" para el número de ocurrencias.

```
# Crear DataFrames para las palabras positivas y negativas actualizadas con el conteo
df_palabras_positivas_actualizadas = pd.DataFrame({'palabra': list(conteo_palabras_positivas.keys()), 'conteo': list(conteo_palabras_positivas.values())})
df_palabras_negativas_actualizadas = pd.DataFrame({'palabra': list(conteo_palabras_negativas.keys()), 'conteo': list(conteo_palabras_negativas.values())})
```

Impresión de los DataFrames

Esta parte del código imprime los DataFrames actualizados que contienen las palabras positivas y negativas junto con sus conteos.

Uso: Proporciona una salida legible para visualizar las palabras y sus ocurrencias.

Método: Utiliza la función `print()` para mostrar los DataFrames en la consola. Cada DataFrame se imprime con su nombre y contenido.

```
# Imprimir los DataFrames
print("DataFrame de palabras positivas actualizadas:")
print(df_palabras_positivas_actualizadas)

print("\nDataFrame de palabras negativas actualizadas:")
print(df_palabras_negativas_actualizadas)
```

Iteración sobre Cada Fila del DataFrame y Obtención del Texto del Tweet

Este bucle itera sobre cada fila del DataFrame `df_tweets` y esta otra parte obtiene el texto del tweet limpio de la columna "tweets limpios" de la fila actual.

Uso: Permite procesar cada tweet individualmente para su análisis.

Método: Utiliza el método `iterrows()` de pandas para iterar sobre cada fila del DataFrame. En cada iteración, `index` es el índice de la fila y `row` es un objeto que representa la fila actual.

Uso: Proporciona el texto del tweet para su posterior procesamiento.

Método: Accede al valor de la columna "tweets limpios" en la fila actual utilizando `row["tweets limpios"]` y lo asigna a la variable `tweet_text`.

```
# Iterar sobre cada fila del DataFrame
for index, row in df_tweets.iterrows():
    # Obtener el texto del tweet de la columna "texto"
    tweet_text = row["tweets limpios"]
```

División del Texto en Palabras y Iteración sobre Cada Palabra del Tweet

Este paso divide el texto del tweet en palabras individuales.

Uso: Facilita el análisis y procesamiento de cada palabra individualmente.

Método: Utiliza el método `split()` para dividir el texto del tweet en palabras individuales. Por defecto, divide el texto en palabras separadas por espacios en blanco y devuelve una lista de palabras.

El bucle itera sobre cada palabra en el tweet.

Uso: Permite realizar operaciones específicas en cada palabra del tweet.

Método: Itera sobre cada elemento de la lista palabras que contiene las palabras individuales del tweet. En cada iteración, imprime la palabra actual. Esto podría ser reemplazado por cualquier otra operación que desees realizar en cada palabra.

```
# Dividir el texto en palabras
palabras = tweet_text.split()

# Iterar sobre cada palabra
for palabra in palabras:
    print(palabra)
```

Llenado de Valores Faltantes en la Columna "conteo"

Esta parte del código llena los valores faltantes en la columna "conteo" de los DataFrames `df_negativa` y `df_positiva` con ceros.

Uso: Asegura que todos los DataFrames tengan valores iniciales de conteo antes de actualizarlos con las ocurrencias de palabras en los tweets.

Método: Utiliza el método `fillna()` de pandas para rellenar los valores faltantes en la columna "conteo" con ceros. El argumento `inplace=True` actualiza directamente los DataFrames originales.

```
df_negativa["conteo"].fillna(0, inplace=True)
df_positiva["conteo"].fillna(0, inplace=True)
```

Actualización de los DataFrames con el Conteo de Palabras

Este bloque del código se encarga de actualizar los DataFrames `df_palabras_negativas_actualizadas` y `df_palabras_positivas_actualizadas` con el conteo de ocurrencias de palabras negativas y positivas respectivamente, utilizando los tweets limpios.

Uso: Permite contar cuántas veces aparece cada palabra positiva y negativa en los tweets para su posterior análisis.

Método: Itera sobre cada fila del DataFrame `df_tweets` y divide el texto de cada tweet en palabras individuales. Luego, verifica si cada palabra está presente en las listas de palabras positivas y negativas. Si lo está, incrementa el conteo correspondiente en los DataFrames actualizados.

```
for index, row in df_tweets.iterrows():

    tweet_text = row["tweets limpios"]

    palabras = tweet_text.split()

    for palabra in palabras:
        if palabra in df_negativa["palabra"].values:
            df_palabras_negativas_actualizadas.loc[df_palabras_negativas_actualizadas["palabra"] == palabra, "conteo"] += 1

        if palabra in df_positiva["palabra"].values:
            df_palabras_positivas_actualizadas.loc[df_palabras_positivas_actualizadas["palabra"] == palabra, "conteo"] += 1
```

Impresión de los DataFrames Actualizados

Esta parte del código imprime las primeras filas de los DataFrames actualizados que contienen las palabras positivas y negativas junto con sus conteos.

Uso: Proporciona una salida legible para visualizar las palabras y sus conteos después de la actualización.

Método: Utiliza la función `print()` para mostrar los DataFrames actualizados en la consola.

```
print("Primeras filas de BibliotecaFinalNegativa.xlsx:")
print(df_palabras_negativas_actualizadas)

print("\nPrimeras filas de BibliotecaFinalPositiva.xlsx:")
print(df_palabras_positivas_actualizadas)
```

Guardado de los DataFrames en Archivos Excel

Esta parte del código guarda los DataFrames actualizados en archivos Excel para su posterior análisis o referencia.

Uso: Permite almacenar los resultados del análisis de las palabras positivas y negativas en un formato fácil de compartir y manipular.

Método: Utiliza el método `to_excel()` de pandas para guardar los DataFrames en archivos Excel. El argumento `index=False` evita que se incluyan los índices de fila en los archivos Excel resultantes.

```
# Guardar los DataFrames actualizados en archivos Excel
df_palabras_positivas_actualizadas.to_excel("palabras_positivasX_conteo.xlsx", index=False)
df_palabras_negativas_actualizadas.to_excel("palabras_negativasX_conteo.xlsx", index=False)
```

Filtrado y Ordenamiento de DataFrames

Estos pasos filtran las palabras positivas y negativas para incluir solo aquellas con un conteo mayor que cero.

Uso: Elimina las palabras que no tienen ocurrencias en los tweets para centrarse en las más relevantes.

Método: Utiliza la función de indexación booleana para seleccionar solo las filas donde el conteo es mayor que cero.

Estos pasos ordenan los DataFrames filtrados por conteo en orden descendente.

Uso: Asegura que las palabras más comunes estén en la parte superior de los gráficos.

Método: Utiliza el método `sort_values()` de pandas para ordenar los DataFrames por la columna "conteo" en orden descendente.

```
df_positiva_filtrado = df_palabras_positivas_actualizadas[df_palabras_positivas_actualizadas["conteo"] > 0]
df_negativa_filtrado = df_palabras_negativas_actualizadas[df_palabras_negativas_actualizadas["conteo"] > 0]
df_positiva_filtrado = df_positiva_filtrado.sort_values(by="conteo", ascending=False)
df_negativa_filtrado = df_negativa_filtrado.sort_values(by="conteo", ascending=False)
```

Selección de las 10 Palabras más Comunes

Estos pasos seleccionan las 10 palabras más comunes de los DataFrames filtrados.

Uso: Limita el número de palabras mostradas en los gráficos para mantener la visualización clara.

Método: Utiliza el método `head()` de pandas para seleccionar las primeras 10 filas de cada DataFrame.

```
top_palabras_positivas = df_positiva_filtrado.head(10)
top_palabras_negativas = df_negativa_filtrado.head(10)
```

Creación de Gráficos de Barras Horizontales

Estos pasos crean los gráficos de barras horizontales que muestran las palabras más comunes en los DataFrames `df_positiva` y `df_negativa`.

Uso: Proporciona una visualización clara y comparativa de las palabras más relevantes en cada categoría.

Método:

`plt.subplots()`: Crea una figura y un conjunto de subgráficos.

`axs.barh()`: Crea gráficos de barras horizontales para las palabras y sus conteos.

`axs.set_title()`, `axs.set_xlabel()`, `axs.set_ylabel()`: Establece títulos y etiquetas de ejes para cada gráfico.

`plt.tight_layout()`: Ajusta automáticamente los espacios entre subgráficos para mejorar la legibilidad.

`plt.show()`: Muestra los gráficos.

```
fig, axs = plt.subplots(2, 1, figsize=(10, 12))

axs[0].barh(top_palabras_positivas["palabra"], top_palabras_positivas["conteo"], color='green')
axs[0].set_title('Palabras más comunes en df_positiva')
axs[0].set_xlabel('conteo')
axs[0].set_ylabel('palabra')

axs[1].barh(top_palabras_negativas["palabra"], top_palabras_negativas["conteo"], color='red')
axs[1].set_title('Palabras más comunes en df_negativa')
axs[1].set_xlabel('conteo')
axs[1].set_ylabel('palabra')

plt.tight_layout()

plt.show()
```

Creación de Columna "sentimiento" en el DataFrame

Se crea una nueva columna llamada "sentimiento" en el DataFrame `df_tweets` para almacenar las etiquetas de sentimiento de los tweets.

Uso: Permitirá etiquetar manualmente los tweets como positivos, negativos o neutrales en función de las palabras que contengan.

Método: Utiliza la asignación directa para inicializar todos los tweets con el valor "neutral".

```
df_tweets['sentimiento'] = 'neutral'
```

Etiquetado de Tweets como Positivos o Negativos

Itera sobre las palabras positivas y negativas y etiqueta los tweets que contienen estas palabras como positivos o negativos, respectivamente.

Uso: Asigna etiquetas de sentimiento a los tweets basándose en las palabras que aparecen en ellos.

Método: Utiliza el método `contains()` de pandas para verificar si una palabra está presente en el texto del tweet y luego actualiza la columna "sentimiento" en consecuencia.

```
for palabra in palabras_positivas:
    df_tweets.loc[df_tweets['tweets limpios'].str.contains(palabra), 'sentimiento'] = 'positivo'

# Etiquetar como negativos los tweets que contienen palabras negativas
for palabra in palabras_negativas:
    df_tweets.loc[df_tweets['tweets limpios'].str.contains(palabra), 'sentimiento'] = 'negativo'
```

Preprocesamiento de Texto y Creación de Matriz de Términos de Documentos (DTM)

Se convierten los textos de los tweets en una matriz numérica que representa la frecuencia de cada palabra en cada tweet.

Uso: Prepara los datos de texto para ser utilizados como entrada para el modelo de clasificación.

Método: Se utiliza CountVectorizer() de scikit-learn para convertir los textos de los tweets en una matriz de términos de documentos (DTM) donde cada fila representa un tweet y cada columna representa una palabra única.

```
vectorizer = CountVectorizer()  
X = vectorizer.fit_transform(df_tweets['tweets limpios'])  
y = df_tweets['sentimiento']
```

División de Datos en Entrenamiento y Prueba

Divide los datos en conjuntos de entrenamiento y prueba para evaluar el rendimiento del modelo.

Uso: Proporciona datos independientes para entrenar y evaluar el modelo.

Método: Utiliza train_test_split() de scikit-learn para dividir la matriz de características (X) y las etiquetas (y) en conjuntos de entrenamiento y prueba.

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
```

Inicialización y Entrenamiento del Clasificador Naive Bayes Multinomial

Se inicializa y entrena un modelo de clasificación Naive Bayes multinomial.

Uso: Se utiliza para clasificar los tweets en categorías de sentimiento.

Método: Se utiliza MultinomialNB() de scikit-learn para inicializar el clasificador y luego se entrena con los datos de entrenamiento.

```
classifier = MultinomialNB()  
classifier.fit(X_train, y_train)
```

Evaluación del Clasificador

Evalúa el rendimiento del clasificador en el conjunto de datos de prueba.

Uso: Proporciona una medida de qué tan bien el clasificador puede predecir el sentimiento de los tweets.

Método: Se utiliza el método score() de scikit-learn para calcular la exactitud del modelo en los datos de prueba.

```
accuracy = classifier.score(X_test, y_test)  
print("Exactitud del clasificador:", accuracy)
```

Etiquetado de Tweets como Positivos o Negativos

Itera sobre las palabras positivas y negativas para etiquetar los tweets correspondientes como positivos o negativos.

Uso: Asigna etiquetas de sentimiento a los tweets en función de las palabras que contienen.

Método: Utiliza el método `contains()` de pandas para verificar si una palabra está presente en el texto del tweet y luego actualiza la columna "sentimiento" en consecuencia.

```
for palabra in palabras_positivas:
    df_tweets.loc[df_tweets['tweets limpios'].str.contains(palabra), 'sentimiento'] = 'positivo'

# Etiquetar como negativos los tweets que contienen palabras negativas
for palabra in palabras_negativas:
    df_tweets.loc[df_tweets['tweets limpios'].str.contains(palabra), 'sentimiento'] = 'negativo'
```

Guardar el DataFrame Actualizado en un Archivo Excel

Guarda el DataFrame actualizado con las etiquetas de sentimiento en un archivo Excel.

Uso: Permite almacenar los datos etiquetados para su posterior análisis o referencia.

Método: Utiliza el método `to_excel()` de pandas para guardar el DataFrame en un archivo Excel, con la opción `index=False` para evitar guardar el índice de las filas en el archivo.

```
df_tweets.to_excel("tweets_con_sentimientoX.xlsx", index=False)
```

Conteo de la Cantidad de Tweets por Etiqueta de Sentimiento

Propósito: Calcula el número de tweets para cada etiqueta de sentimiento.

Uso: Proporciona información sobre la distribución de los sentimientos en el conjunto de tweets.

Método: Utiliza el método `value_counts()` de pandas para contar las ocurrencias de cada valor único en la columna 'sentimiento'.

```
# Contar la cantidad de tweets por cada etiqueta de sentimiento
sentimiento_counts = df_tweets['sentimiento'].value_counts()

# Obtener los nombres de las etiquetas y sus conteos
etiquetas = sentimiento_counts.index
conteos = sentimiento_counts.values
```

Creación del Gráfico de Barras

Visualiza la distribución de los sentimientos en los tweets utilizando un gráfico de barras.

Uso: Permite comparar la cantidad de tweets para cada etiqueta de sentimiento.

Método:

`plt.figure()`: Crea una nueva figura para el gráfico especificando el tamaño.

`plt.bar()`: Crea el gráfico de barras con etiquetas y conteos, utilizando colores diferentes para cada sentimiento.

`plt.xlabel()`: Establece el nombre del eje x como "Sentimiento".

`plt.ylabel()`: Establece el nombre del eje y como "Cantidad de Tweets".

`plt.title()`: Establece el título del gráfico como "Distribución de Sentimientos en los Tweets".

`plt.show()`: Muestra el gráfico.

```
# Crear el gráfico de barras
plt.figure(figsize=(8, 6))
plt.bar(etiquetas, conteos, color=['green', 'red', 'blue'])
plt.xlabel('Sentimiento')
plt.ylabel('Cantidad de Tweets')
plt.title('Distribución de Sentimientos en los Tweets')
plt.show()
```

Reducción de Dimensionalidad con t-SNE

Reduce la dimensionalidad de la matriz de términos de documentos (DTM) a dos dimensiones utilizando t-SNE.

Uso: Permite visualizar la estructura de los datos en un espacio bidimensional, preservando las relaciones entre los puntos.

Método: Se utiliza TSNE() de scikit-learn para inicializar el modelo t-SNE con dos componentes y luego se ajusta a la matriz de características X.

```
# Reducción de dimensionalidad con t-SNE
tsne = TSNE(n_components=2, random_state=42)
X_tsne = tsne.fit_transform(X.toarray())
```

Creación del Gráfico de Dispersión

Visualiza la distribución de sentimientos en los tweets en un espacio bidimensional utilizando un gráfico de dispersión.

Uso: Permite observar patrones o agrupaciones entre los tweets en función de su sentimiento.

Método: Utiliza plt.figure() para crear una nueva figura para el gráfico especificando el tamaño. Itera sobre los colores y las etiquetas de sentimiento definidos en el diccionario colors.

Utiliza plt.scatter() para crear el gráfico de dispersión con las coordenadas transformadas por t-SNE, asignando un color diferente a cada sentimiento.

plt.xlabel() y plt.ylabel() establecen los nombres de los ejes x e y como "Dimensión 1 (t-SNE)" y "Dimensión 2 (t-SNE)" respectivamente.

plt.title() establece el título del gráfico como "Distribución de Sentimientos en los Tweets (t-SNE)".

plt.legend() muestra la leyenda del gráfico.

plt.show(): Muestra el gráfico.

```
# Crear el gráfico de dispersión
plt.figure(figsize=(10, 8))
colors = {'positivo': 'green', 'negativo': 'red', 'neutral': 'blue'}
for sentimiento, color in colors.items():
    indices = df_tweets['sentimiento'] == sentimiento
    plt.scatter(X_tsne[indices, 0], X_tsne[indices, 1], c=color, label=sentimiento, alpha=0.5)
plt.xlabel('Dimensión 1 (t-SNE)')
plt.ylabel('Dimensión 2 (t-SNE)')
plt.title('Distribución de Sentimientos en los Tweets (t-SNE)')
plt.legend()
plt.show()
```

Predicciones y Cálculo de Precisión

Realiza predicciones sobre los datos de prueba y calcula la precisión del clasificador.

Uso: Evalúa qué tan bien el clasificador puede predecir el sentimiento correcto de los tweets.

Método: `classifier.predict()`: Realiza predicciones sobre los datos de prueba.

`accuracy_score()`: Calcula la precisión comparando las etiquetas verdaderas (`y_test`) con las predicciones.

```
# Realizar predicciones y calcular la precisión
predictions = classifier.predict(X_test)
accuracy = accuracy_score(y_test, predictions)
print("Exactitud del clasificador:", accuracy)
```

Cálculo de la Matriz de Confusión y el Reporte de Clasificación

Calcula la matriz de confusión y el reporte de clasificación para evaluar el rendimiento del clasificador.

Uso: Proporciona información detallada sobre los aciertos y errores del clasificador en la clasificación de los tweets.

Método: `confusion_matrix()`: Calcula la matriz de confusión, que muestra el número de predicciones correctas e incorrectas para cada clase.

`classification_report()`: Genera un reporte de clasificación que incluye precision, recall, f1-score y support para cada clase, así como la precisión global y el promedio ponderado de estas métricas.

```
# Calcular la matriz de confusión y el reporte de clasificación
confusion_mat = confusion_matrix(y_test, predictions)
print("\nMatriz de confusión:")
print(confusion_mat)

class_report = classification_report(y_test, predictions)
print("\nReporte de clasificación:")
print(class_report)
```

Definición y Ajuste del Modelo

Define una función para ajustar un modelo de clasificación Naive Bayes multinomial utilizando GridSearchCV para encontrar los mejores parámetros.

Uso: Permite seleccionar automáticamente los mejores parámetros del clasificador para mejorar su rendimiento.

Método: GridSearchCV(): Realiza una búsqueda exhaustiva sobre una cuadrícula de parámetros especificada.

fit(): Entrena el modelo con la búsqueda en la cuadrícula.

best_estimator_: Obtiene el mejor clasificador encontrado durante la búsqueda.

```
def ajustar_modelo(X_train, y_train):  
    # Definir los parámetros a ajustar  
    parametros = {'alpha': [0.5, 1.0, 1.5]}  
    # Inicializar el clasificador y realizar la búsqueda en la cuadrícula  
    classifier = MultinomialNB()  
    grid_search = GridSearchCV(classifier, parametros, cv=5, scoring='accuracy')  
    grid_search.fit(X_train, y_train)  
    # Seleccionar el mejor clasificador encontrado  
    best_classifier = grid_search.best_estimator_  
    return best_classifier  
  
# Ajustar el modelo  
best_classifier = ajustar_modelo(X_train, y_train)
```

Evaluación del Modelo

Realiza predicciones sobre los datos de prueba utilizando el mejor clasificador encontrado y evalúa su rendimiento.

Uso: Proporciona una medida de la precisión y otras métricas de evaluación como la matriz de confusión y el reporte de clasificación.

Método: `predict()`: Realiza predicciones sobre los datos de prueba utilizando el mejor clasificador.

`accuracy_score()`: Calcula la precisión comparando las etiquetas verdaderas con las predicciones.

`confusion_matrix()`: Calcula la matriz de confusión para evaluar el rendimiento del modelo.

`classification_report()`: Genera un reporte de clasificación detallado con métricas como precision, recall, f1-score y support.

```
# Realizar predicciones y calcular la precisión
predictions = best_classifier.predict(X_test)
accuracy = accuracy_score(y_test, predictions)
print("Exactitud del clasificador:", accuracy)

# Calcular la matriz de confusión y el reporte de clasificación
confusion_mat = confusion_matrix(y_test, predictions)
print("\nMatriz de confusión:")
print(confusion_mat)

class_report = classification_report(y_test, predictions)
print("\nReporte de clasificación:")
print(class_report)
```

Visualización de la Matriz de Confusión

Visualiza la matriz de confusión para evaluar el rendimiento del modelo.

Uso: Proporciona una representación gráfica de las predicciones correctas e incorrectas del modelo.

Método: `plt.imshow()`: Muestra la matriz de confusión como una imagen.

`plt.colorbar()`: Agrega una barra de color para indicar los valores de la matriz.

`plt.text()`: Agrega el valor de cada celda de la matriz como texto en la gráfica.

```
# Imprimir la matriz de confusión utilizando matplotlib
plt.figure(figsize=(8, 6))
plt.imshow(confusion_mat, interpolation='nearest', cmap=plt.cm.Blues)
plt.title('Matriz de Confusión')
plt.colorbar()

tick_marks = np.arange(len(np.unique(y_test)))
plt.xticks(tick_marks, np.unique(y_test))
plt.yticks(tick_marks, np.unique(y_test))

thresh = confusion_mat.max() / 2.
for i, j in itertools.product(range(confusion_mat.shape[0]), range(confusion_mat.shape[1])):
    plt.text(j, i, format(confusion_mat[i, j], 'd'),
             horizontalalignment="center",
             color="white" if confusion_mat[i, j] > thresh else "black")

plt.tight_layout()
plt.ylabel('Etiqueta verdadera')
plt.xlabel('Etiqueta predicha')
plt.show()
```

Impresión del Reporte de Clasificación

Imprime el reporte de clasificación que incluye precision, recall, f1-score y support para cada clase.

Uso: Proporciona una descripción detallada del rendimiento del modelo en la clasificación de cada clase.

Método: `classification_report()`: Genera el reporte de clasificación con métricas de evaluación detalladas.

```
class_report = classification_report(y_test, predictions)
print("\nReporte de clasificación:")
print(class_report)
```

En el caso de querer cargar otros tweets para su análisis, se deberán subir de la misma forma en que se especificó anteriormente y se tienen que actualizar los nombres de igual forma, así como se señala en la parte de **“Cargar Archivos para Análisis”**.